

KNOWTIFY

Fase de Diseño — Actividades y Evidencias

Documentación del proceso de diseño con capturas de pantalla

Proyecto: KNOWTIFY (edu-grade-mocavi)

Documento: Matriz de actividades de la fase de diseño

Fecha: 21 de mayo de 2026

Versión: 1.0

Introducción

Este documento detalla las actividades técnicas ejecutadas durante la fase de diseño y construcción de la plataforma KNOWTIFY, acompañadas de su evidencia visual (capturas de pantalla de la aplicación en ejecución y del código fuente real). Las actividades se organizan en cinco componentes: Núcleo de IA, Infraestructura de Datos, Interfaz de Usuario, Comunicación y Soporte, y Seguridad y Despliegue. Al final se incluyen otras actividades relevantes del proyecto.

Matriz de actividades

Componente	Actividad técnica	Descripción y resultado
Núcleo de IA (Backend)	Integración de Gemini Flash	Configurar la API para visión computacional y lectura de exámenes manuscritos.
Núcleo de IA (Backend)	Vinculación elástica (Fuzzy Matching)	Asociar notas a estudiantes con nombres poco legibles.
Núcleo de IA (Backend)	Lógica de calificación masiva	Comparador de respuestas con clave JSON sobre PDFs e imágenes.
Infraestructura de Datos	Modelado en PostgreSQL	Tablas de usuarios, dimensiones Saber/Hacer/Ser e historial de promedios.
Infraestructura de Datos	Endpoints con FastAPI	Comunicación asíncrona para subida de archivos y consulta de notas.
Interfaz de Usuario	Dashboard del docente	Planilla con edición directa y alertas de color (verde a rojo).
Interfaz de Usuario	Panel del estudiante	Rendimiento con retroalimentación de la IA y promedios.
Interfaz de Usuario	Tema oscuro y responsividad	Tailwind CSS: móviles y protección de la salud visual.
Comunicación y Soporte	Implementación de WebSockets	Mensajería en tiempo real docente-estudiante.
Comunicación y Soporte	Tutor IA 24/7	Chat con soporte Markdown para explicaciones académicas.
Seguridad y Despliegue	Middleware de autenticación	Acceso por roles (teacher/student) y protección

		de datos.
Seguridad y Despliegue	Despliegue en la nube	Entorno de producción (Render) para pruebas piloto.

Componente 1 — Núcleo de IA (Backend)

Actividad 1. Integración de Gemini (visión computacional)

Objetivo: Configurar la API para el procesamiento de visión computacional y la lectura de exámenes manuscritos.

Implementación técnica: Se integró el cliente de Google Gemini (modelo Flash, autodetectado en el arranque con fallback a gemini-2.0-flash). El recorte del nombre manuscrito de cada hoja se envía como imagen al modelo junto con la lista de la clase, con tiempo de espera y reintentos controlados.

Evidencia:

```
1 client = genai.Client(api_key=os.getenv("GEMINI_API_KEY"))
2
3 def get_active_model_name() -> str:
4     for m in client.models.list():
5         if "generateContent" in m.supported_actions and "flash" in m.name:
6             return m.name # auto-detecta el modelo Flash disponible
7     return "gemini-2.0-flash" # fallback
8
9 MODEL_NAME = get_active_model_name()
10
11 # Lectura de nombres manuscritos: se envían los recortes + la lista de la clase
12 contents = []
13 for n, (_i, img) in enumerate(valid, start=1):
14     contents.append(f"Hoja {n}:")
15     contents.append(types.Part.from_bytes(data=img, mime_type="image/png"))
16 resp = await asyncio.to_thread(client.models.generate_content,
17                                model=MODEL_NAME, contents=contents)
```

Figura 1. Integración del cliente Gemini y envío de los recortes de nombre a la IA de visión.

Actividad 2. Algoritmo de vinculación elástica (Fuzzy Matching)

Objetivo: Programar el motor de vinculación elástica para asociar notas a estudiantes con nombres poco legibles.

Implementación técnica: La identificación combina la normalización del texto (minúsculas, sin acentos ni espacios sobrantes) con el emparejamiento contra una lista cerrada de estudiantes de la clase. Así tolera variaciones de escritura (tildes, mayúsculas); si no hay coincidencia clara, la hoja queda para asignación manual como respaldo.

Evidencia:

```

1 def _normalize_name(s: str) -> str:
2     """minúsculas, sin acentos ni espacios extra – para emparejar nombres."""
3     import unicodedata
4     s = unicodedata.normalize("NFKD", s or "").encode("ascii", "ignore").decode()
5     return " ".join(s.lower().split())
6
7 # La IA devuelve el nombre; se empareja contra la lista cerrada de la clase
8 by_norm = {_normalize_name(s.full_name): s for s in roster}
9 out = [None] * len(name_imgs)
10 for n, (i, _img) in enumerate(valid, start=1):
11     nombre = name_by_hoja.get(n)
12     if nombre:
13         out[i] = by_norm.get(_normalize_name(nombre)) # tolera tildes/mayúsculas
14 return out

```

Figura 2. Normalización de nombres y emparejamiento contra la lista de la clase.

Actividad 3. Lógica de calificación masiva (OMR)

Objetivo: Desarrollar el comparador de respuestas basado en la clave JSON para procesar PDFs e imágenes.

Implementación técnica: El motor OMR (OpenCV) normaliza la imagen mediante cuatro fiduciales, mide el relleno de cada burbuja A/B/C/D y la compara con la clave en formato JSON. Acepta PDF e imágenes y calcula la nota en escala 1.0–5.0. La misma geometría genera la plantilla imprimible y lee las marcas.

Evidencia:

```

1 # La MISMA geometría genera la plantilla imprimible y lee las marcas
2 SHEET_W, SHEET_H = 1000, 1414 # lienzo normalizado (A4)
3 GRID_TOP, ROW_H, BUBBLE_R = 380, 50, 16
4
5 def read_marks(file_bytes, content_type, num_q, num_opts=4):
6     pages = _load_pages(file_bytes, content_type) # PDF o imagen
7     for gray in pages:
8         warped, aligned = _warp_to_canvas(gray) # corrige perspectiva
9         for q, opts in bubble_centers(num_q).items():
10             ratios = [_fill_ratio(bin_img, cx, cy, BUBBLE_R) for cx, cy in opts]
11             answers[q] = OPTION_LETTERS[int(np.argmax(ratios))]
12
13 # Comparación con la clave (JSON) y nota en escala 1.0 - 5.0
14 is_ok = sel is not None and sel == key_norm.get(q)
15 score = round(correct / num_q * 4 + 1, 2)

```

Figura 3. Lectura de marcas con OpenCV y cálculo de la nota a partir de la clave JSON.

Componente 2 — Infraestructura de Datos

Actividad 4. Modelado en PostgreSQL

Objetivo: Crear las tablas de usuarios, las dimensiones Saber/Hacer/Ser y el historial de promedios académicos.

Implementación técnica: El modelo de datos se compone de ocho entidades implementadas con SQLAlchemy (UUID como clave primaria y JSONB para respuestas y claves). Las notas se almacenan por dimensión (Saber/Hacer/Ser) y por periodo, con la nota final y el nivel de desempeño calculados.

Evidencia:

```
1 class Student(Base):
2     __tablename__ = "students"
3     id = Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
4     full_name = Column(String, nullable=False)
5     document_id = Column(String, unique=True, index=True)
6     grade = Column(String, default="11", index=True)
7     grupo = Column(String, default="1", index=True)
8
9 class Grade(Base):
10     __tablename__ = "grades"
11     period_id = Column(Integer, nullable=False, index=True)
12     score_saber = Column(Numeric(3, 2), default=1.0)
13     score_hacer = Column(Numeric(3, 2), default=1.0)
14     score_ser = Column(Numeric(3, 2), default=1.0)
15     final_period_score = Column(Numeric(3, 2))
16     performance_level = Column(String) # SUPERIOR | ALTO | BÁSICO | BAJO
```

Figura 4. Definición de las entidades Student y Grade (dimensiones y nota final) con SQLAlchemy.

Actividad 5. Endpoints con FastAPI

Objetivo: Construir los puntos de comunicación asíncronos para gestionar la subida de archivos y la consulta de notas.

Implementación técnica: La API expone alrededor de 30 endpoints REST. Las operaciones intensivas de E/S (lectura OMR, llamadas a la IA) se ejecutan de forma asíncrona (async/await y ejecución en hilos) para no bloquear el servidor. FastAPI genera además documentación interactiva (Swagger) de forma automática.

Evidencia:

```

1 @app.post("/upload-exams-omr")
2 @limiter.limit("30/minute")
3 async def upload_exams_omr(file: UploadFile = File(...),
4                             answer_key: str = Form(...),
5                             grade=Form(None), grupo=Form(None),
6                             period_id=Form(None),
7                             current_user: dict = Depends(auth.get_current_teacher),
8                             db: Session = Depends(database.get_db)):
9     content = await file.read()
10    # Lectura OMR en un hilo para no bloquear el bucle asíncrono
11    pages = await asyncio.to_thread(omr.read_marks, content, ct, num_q, 4)
12    identified = await _identify_names_by_ai([p["name_img"] for p in pages], roster)
13    ...
14    return {"exam_id": str(db_exam.id), "analisis": resultados}

```

Figura 5. Endpoint asíncrono de calificación OMR con subida de archivos y validación por rol.

MOCABI EDU-GRADE API 1.0.0 OAS 3.1
openapi.json

[Authorize](#)

default		^
POST	/login/teacher Login Teacher	▼
POST	/login/student Login Student	▼
POST	/newsletter Newsletter Subscribe	▼
GET	/newsletter/subscribers Newsletter List	▼
GET	/student/dashboard Student Dashboard	▼
GET	/students/ Get Students	▼
POST	/students/ Create Student	▼
POST	/students/bulk Bulk Import Students	▼
DELETE	/students/{student_id} Delete Student	▼
GET	/grades/ Get Grades	▼
POST	/grades/ Create Or Update Grade	▼
POST	/upload-exams Upload Exams	▼

Figura 6. Documentación interactiva de la API (Swagger UI) generada automáticamente por FastAPI.

Componente 3 — Interfaz de Usuario (Frontend)

Actividad 6. Dashboard del docente

Objetivo: Diseñar la planilla inteligente con edición directa y un sistema de alertas de colores (verde a rojo).

Implementación técnica: El tablero presenta tarjetas de resumen (estudiantes, promedio y riesgo) y una tabla de notas por dimensión, con el nivel de desempeño codificado por color (de SUPERIOR/ALTO en verde a BAJO en rojo), edición directa de notas y exportación a Excel y PDF.

Evidencia:

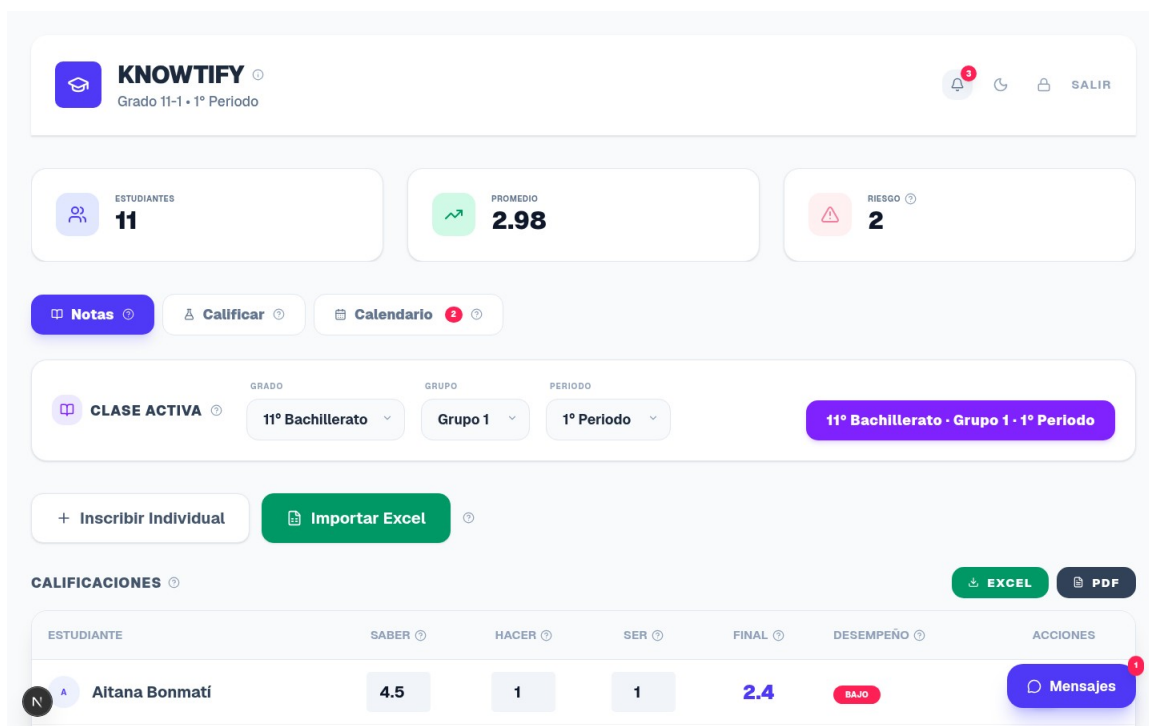


Figura 7. Planilla del docente: tarjetas de resumen, notas por dimensión y alertas de color.

Actividad 7. Panel del estudiante

Objetivo: Crear la vista de rendimiento con retroalimentación generada por la IA y promedios generales.

Implementación técnica: El panel muestra el promedio general y el nivel, una gráfica de evolución del promedio por periodo (recharts), los próximos eventos del calendario y el detalle por periodo con la retroalimentación de cada examen.

Evidencia:

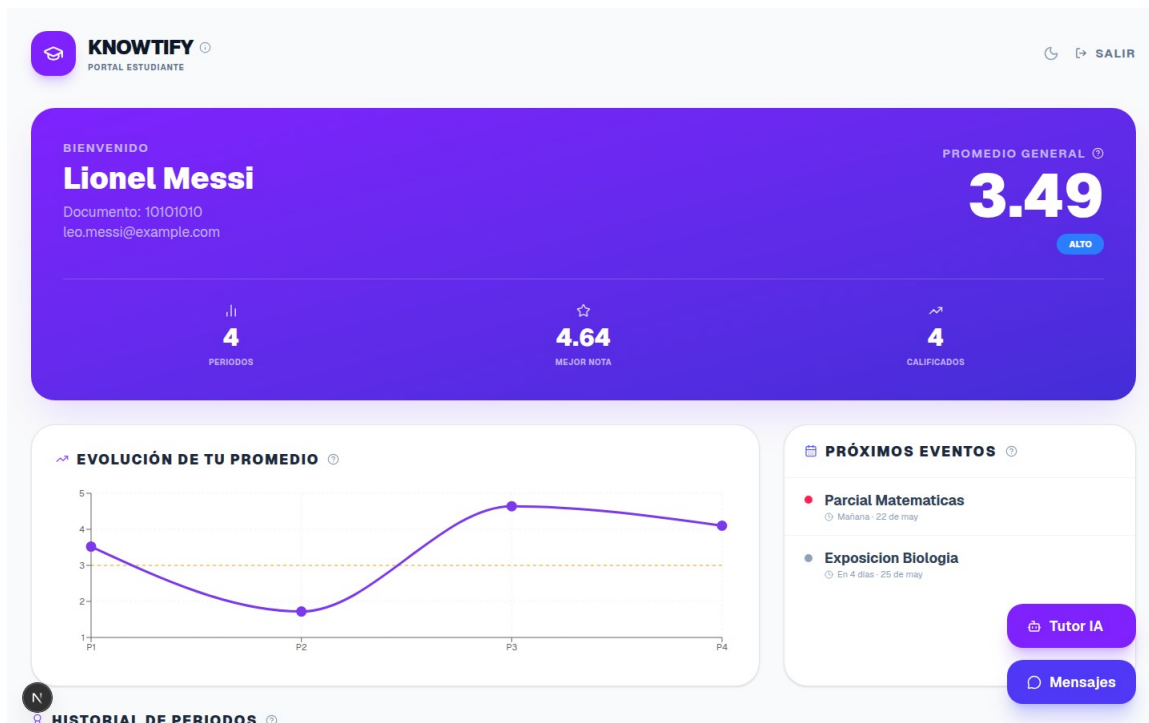


Figura 8. Panel del estudiante: promedio general, evolución por periodo y próximos eventos.

Actividad 8. Tema oscuro y responsividad

Objetivo: Implementar Tailwind CSS para asegurar que la plataforma funcione en móviles y proteja la salud visual.

Implementación técnica: La interfaz usa Tailwind CSS v4 con modo claro/oscuro conmutable (next-themes) y un diseño responsive que adapta el contenido a pantallas pequeñas, cuidando el contraste y la legibilidad.

Evidencia:

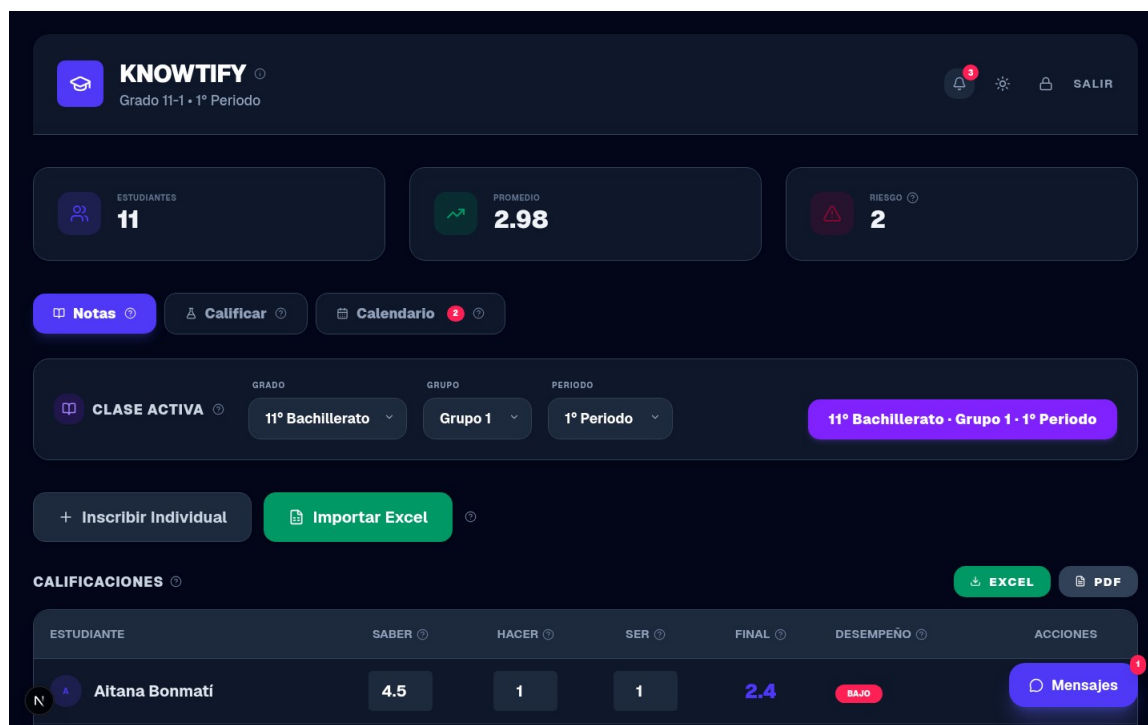


Figura 9. Modo oscuro del tablero del docente (protección de la salud visual).

Adaptación a dispositivos móviles (vista responsive a 390 px de ancho):



Figura 10. Vista responsive en móvil: página de inicio (izq.) y panel del estudiante (der.).

Componente 4 — Comunicación y Soporte

Actividad 9. Implementación de WebSockets

Objetivo: Desarrollar el sistema de mensajería en tiempo real para el widget de comunicación docente–estudiante.

Implementación técnica: Se implementó un endpoint WebSocket autenticado por JWT (enviado en la URL). El servidor mantiene un registro de conexiones activas y empuja cada mensaje al receptor en tiempo real. El cliente abre la conexión y conserva un sondeo (polling) cada 30 s como respaldo.

Evidencia:

```
1 ws_connections: dict = {}          # user_id -> conexión activa
2
3 @app.websocket("/ws/{token}")
4 async def websocket_endpoint(websocket: WebSocket, token: str):
5     payload = auth.verify_token(token)      # autenticación por JWT
6     user_id = payload.get("sub")
7     await websocket.accept()
8     ws_connections[user_id] = websocket
9     try:
10         while True:
11             await websocket.receive_text()  # mantiene viva la conexión
12         except WebSocketDisconnect:
13             ws_connections.pop(user_id, None)
14
15 async def _ws_notify(receiver_id, msg):    # empuja el mensaje al receptor
16     ws = ws_connections.get(receiver_id)
17     if ws:
18         await ws.send_json({"sender_id": msg.sender_id, "content": msg.content})
```

Figura 11. Endpoint WebSocket autenticado por JWT y notificación en tiempo real (backend).

```
1 // Conexión en tiempo real al backend con el token JWT en la URL
2 const wsUrl = `${process.env.NEXT_PUBLIC_API_URL.replace("http", "ws")}/ws/${token}`;
3 ws = new WebSocket(wsUrl);
4 ws.onmessage = () => { loadMessages(sidRef.current, tidRef.current); };
5 ws.onerror = () => {};
6
7 // Respaldo: si el WebSocket falla, se refresca por polling cada 30 s
8 interval = setInterval(
9     () => loadMessages(sidRef.current, tidRef.current), 30000
10 );
11 return () => { clearInterval(interval); ws.close(); };
```

Figura 12. Cliente WebSocket en el frontend con respaldo por polling.

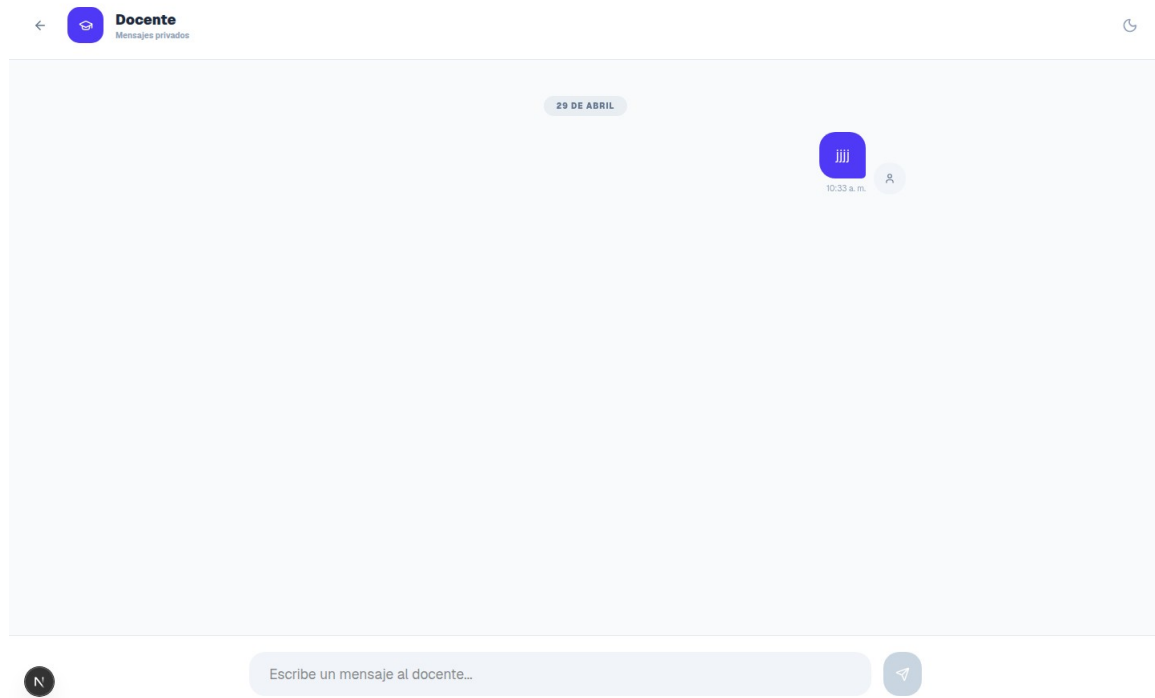


Figura 13. Mensajería entre estudiante y docente en tiempo real.

Actividad 10. Tutor IA 24/7

Objetivo: Programar la interfaz de chat con soporte Markdown para explicaciones académicas automatizadas.

Implementación técnica: El chat se conecta al endpoint /chatbot (Gemini), conservando el historial de la conversación. Las respuestas del tutor se renderizan con formato Markdown (negritas, listas numeradas y viñetas) para una lectura clara.

Evidencia:

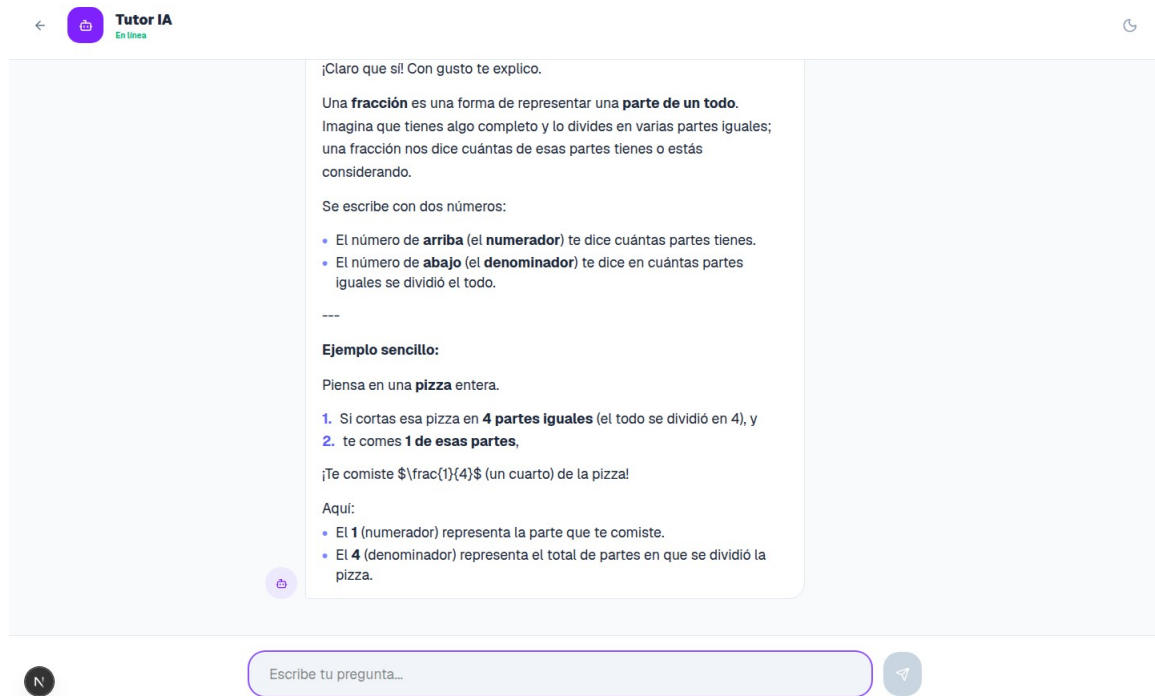


Figura 14. Tutor IA respondiendo una consulta académica con formato Markdown.

Componente 5 — Seguridad y Despliegue

Actividad 11. Middleware de autenticación

Objetivo: Validar el acceso mediante roles (teacher/student) y proteger la integridad de las planillas digitales.

Implementación técnica: Se diseñaron dependencias de FastAPI que verifican el token JWT y el rol del usuario: `get_current_teacher` y `get_current_student` devuelven 403 si el rol no corresponde. Las contraseñas se cifran con `bcrypt` y los endpoints sensibles aplican limitación de tasa.

Evidencia:

```
1 def get_current_user(credentials = Depends(security)) -> dict:
2     payload = verify_token(credentials.credentials) # valida y decodifica JWT
3     return payload
4
5 def get_current_teacher(current_user: dict = Depends(get_current_user)) -> dict:
6     if current_user.get("role") != "teacher":
7         raise HTTPException(status_code=403, detail="Acceso restringido a docentes")
8     return current_user
9
10 def get_current_student(current_user: dict = Depends(get_current_user)) -> dict:
11     if current_user.get("role") != "student":
12         raise HTTPException(status_code=403, detail="Acceso restringido a estudiantes")
13     return current_user
```

Figura 15. Dependencias de autenticación por rol (middleware) en el backend.

The screenshot shows a web application interface for a teacher portal. At the top, there is a navigation bar with a link to 'VOLVER AL INICIO' and a user profile icon. Below this, there are two tabs: 'DOCENTE' (selected) and 'ESTUDIANTE'. The main content area features a large white card with a blue header containing a graduation cap icon and the text 'KNOWTIFY PORTAL DEL DOCENTE'. Below the header, there are two input fields: 'Usuario' and 'Contraseña', each with a corresponding icon. At the bottom of the card is a large blue button labeled 'ENTRAR AL SISTEMA'. In the bottom left corner of the page, there is a small circular logo with the letter 'N'.

Figura 16. Inicio de sesión con portales diferenciados por rol (docente/estudiante).

Actividad 12. Despliegue en la nube

Objetivo: Configurar el entorno de producción (Render) para las pruebas piloto con usuarios reales.

Implementación técnica: El despliegue del backend se definió como Blueprint de Render (render.yaml) junto con un Procfile. Las variables sensibles (DATABASE_URL, GEMINI_API_KEY) se inyectan en el panel del proveedor; la clave JWT_SECRET_KEY se autogenera; se define un healthcheck en /docs y el autodespliegue (autoDeploy).

Evidencia:

```
1 # render.yaml – despliegue del backend KNOWTIFY (Blueprint)
2 services:
3   - type: web
4     name: knowtify-backend
5     runtime: python
6     rootDir: edu-grade-mocavi
7     buildCommand: pip install -r requirements.txt
8     startCommand: uvicorn app.main:app --host 0.0.0.0 --port $PORT
9     healthCheckPath: /docs
10    autoDeploy: true
11    envVars:
12      - key: DATABASE_URL      { sync: false }
13      - key: GEMINI_API_KEY    { sync: false }
14      - key: JWT_SECRET_KEY    { generateValue: true }
15 # Procfile: web: uvicorn app.main:app --host 0.0.0.0 --port $PORT
```

Figura 17. Configuración de despliegue en la nube (render.yaml y Procfile).

Otras actividades importantes

Actividad A. Generación de exámenes con IA

Objetivo: Permitir que el docente genere exámenes de opción múltiple a partir de un tema, con su clave de respuestas.

Implementación técnica: El docente indica materia, tema, número de preguntas y dificultad; la IA (Gemini) genera el examen y su clave. La clave se reutiliza directamente en el Lector OMR para calificar.

Evidencia:

The screenshot displays the OMR reader interface. At the top, there's a header with 'Clave marcada: 0/25'. Below it, a grid of 25 questions is shown, each with four options (A, B, C, D). The questions are numbered 1 through 25. Below the grid, there are two buttons: 'Subir hojas escaneadas / foto...' and 'CALIFICAR HOJAS'. At the bottom, there's a section titled 'Generar examen con IA' with a sub-header 'Materia: Matemáticas (cámbiala en el panel de arriba). Genera las preguntas, imprimelas y reutiliza la clave en el calificador OMR.' Below this, there are three input fields: 'Tema (ej. ecuaciones de primer grado)', '10', and 'Media'. A 'GENERAR EXAMEN' button is located below these fields. In the bottom right corner, there's a 'Mensajes' button with a red notification icon.

Figura 18. Generador de exámenes con IA integrado en el calificador.

Actividad B. Plantilla de hoja de respuestas (OMR)

Objetivo: Generar una hoja de respuestas estándar e imprimible, sincronizada con el lector OMR.

Implementación técnica: El sistema produce en PDF la hoja con los cuatro fiduciales de esquina, la caja del nombre (que se recorta para la identificación por IA) y la cuadrícula de respuestas. La misma geometría que dibuja la hoja es la que luego lee las marcas, evitando desincronizaciones.

Evidencia:

KNOWTIFY — Hoja de respuestas

NOMBRE Y APELLIDOS (escribe en letra clara):

RESPUESTAS

1

A

B

C

D

2

A

B

C

D

3

A

B

C

D

4

A

B

C

D

5

A

B

C

D

6

A

B

C

D

7

A

B

C

D

8

A

B

C

D

9

A

B

C

D

10

A

B

C

D

11

A

B

C

D

12

A

B

C

D

13

A

B

C

D

14

A

B

C

D

15

A

B

C

D

16

A

B

C

D

17

A

B

C

D

18

A

B

C

D

19

A

B

C

D

20

A

B

C

D

21

A

B

C

D

22

A

B

C

D

23

A

B

C

D

24

A

B

C

D

25

A

B

C

D

Rellena completamente la burbuja. Escribe tu nombre completo arriba.

Figura 19. Plantilla de hoja de respuestas generada por el sistema (fiduciales, nombre y cuadrícula).

Actividad C. Otras funciones de apoyo

Además de las anteriores, el sistema incorpora funciones de soporte que refuerzan el diseño:

- Exportación de las notas del periodo a Excel (CSV) y PDF desde el tablero del docente.
- Calendario académico con eventos por grado y grupo, visible para el estudiante.
- Suscripción al boletín «Mantente al día» desde el sitio público (con control de duplicados).
- Cambio de contraseña del docente con verificación de la contraseña actual.
- Validación de datos de entrada con Pydantic y limitación de tasa (rate limiting) en los endpoints sensibles.

— *Fin del documento* —